

# Deploying Contracts ERP to Production

Oracle Cloud instance **business2** · 129.146.22.240 · ARM64, 2 OCPU / 12 GB

Commands shown in **red** are the ones you type.

## What runs on the server

| Container           | What it is                                                                      | Ports         |
|---------------------|---------------------------------------------------------------------------------|---------------|
| contracts-app       | The API — image <code>eradwan/contractsapp:1.0.1</code>                         | 5100,<br>8544 |
| erp-mysql-contracts | <b>The production database</b> — volume <code>erp-mysql-volume-contracts</code> | 3308          |
| portainer           | Container management UI                                                         | 9000,<br>8000 |

## Three rules that matter

- 1. Pull before you build.** If you build first, Docker reuses its cache, the image never changes, and you quietly redeploy the previous version. This has already happened once.
- 2. Always build with `--cpuset-cpus 0`.** Compiling the Persistence project needs more than 6 GB and will use both CPU cores. Pinning it to one core keeps the second free so SSH keeps working.
- 3. Never add `--memory` to the build.** A memory cap makes the C# compiler crash with `OutOfMemoryException` half-way through.

### STEP 1 On your Mac

Run the frontend build **only** if you changed something in `client-app`. Otherwise skip straight to the commit.

```
cd client-app && yarn build && cd ..  
git add -A && git commit -m "build DD-M" && git push
```

You no longer need to delete `API/wwwroot` by hand — Vite now clears it automatically (`emptyOutDir: true`). One commit per deploy is enough.

### STEP 2 On the server

Connect and open a `tmux` session, so the build keeps running even if your connection drops:

```
ssh ubuntu@129.146.22.240  
tmux new -s build
```

Leave tmux with **Ctrl-b** then **d**. Come back later with `tmux attach -t build`.

Get the latest code:

```
cd ~/erp-contracts  
git fetch origin && git reset --hard origin/master && git clean -fd  
git log --oneline -1      # must match what you just pushed
```

Build the image (takes a while — this is normal):

```
DOCKER_BUILDKIT=0 sudo docker build -f Dockerfile.vm --cpuset-cpus 0 \
-t eradwan/contractsapp:1.0.1 . 2>&1 | tee ~/diag/build.log
```

Check the build worked and the image is genuinely new, **then** start it:

```
sudo docker images eradwan/contractsapp # the ID must differ from last deploy
docker compose -f docker-compose.vm.yml up -d
docker compose -f docker-compose.vm.yml ps
```

**If the image ID did not change, stop.** The build failed or used old code — deploying will change nothing. Check the end of the log with `tail -20 ~/diag/build.log`.

### STEP 3 Check it actually worked

From any machine — this asks the live site which version it is serving:

```
curl -s http://129.146.22.240:5100/ | grep -o 'index-[A-Za-z0-9_-]*\.js'
```

The filename changes with every frontend build, so it identifies the deployed version exactly. Compare it with the one in your last commit. To look up which commit a filename came from:

```
git log --diff-filter=A --oneline --name-only -- 'API/wwwroot/assets/index-*.js'
```

### Shortcut for frontend-only changes

The compiled frontend is committed into `API/wwwroot`, and the Docker image contains no npm step. So if a release changed **only** frontend files, the backend is identical and no build is needed:

```
cd ~/erp-contracts && git pull
sudo docker cp API/wwwroot/. contracts-app:/app/wwwroot/
```

Live immediately, no restart. **But it is temporary** — the files sit in the running container, not in the image, so the next `up -d` undoes it. Do a proper build afterwards when you have time.

### Things never to do

**Never prune with `--volumes`**. That deletes `erp-mysql-volume-contracts` — the production database. These are safe:

```
sudo docker image prune -f
sudo docker builder prune -f
```

**Use `docker compose` — no hyphen.** The old `docker-compose` (v1) is still installed and crashes with `KeyError: 'ContainerConfig'` when replacing a container, leaving the MySQL container **deleted**. If you ever must use it, run `down` before `up -d`.

## Back up the database before risky work

```
sudo docker exec erp-mysql-contracts \  
mysql_dump -u root -pTmbtc202500 --single-transaction --routines erp_contracts \  
> ~/mysql-backups/contracts-golden/erp_$(date +%F_%H%M).sql
```

## If the server stops responding

**The symptom:** SSH hangs and fails with `Connection timed out during banner exchange`, and Portainer will not load — but the server is not dead. The kernel is still answering; the operating system is starved.

A failing `ping` is normal here and means nothing — Oracle blocks it by default.

**The cause:** a build running without the CPU pin. It needs more than 6 GB and takes over the machine. Closing your SSH window does *not* stop it — Docker keeps building in the background.

### What to do

1. Oracle Cloud Console → Compute → Instances → **business2** → **Reboot**. Make sure the region selector says **US West (Phoenix)** or the instance will not be listed.
2. If it stays on "Stopping" for more than 5 minutes, press Reboot again and tick the **force** option.
3. Once it is back, restore service **without building**:

```
cd ~/erp-contracts  
docker compose -f docker-compose.vm.yml up -d
```

A force reboot is a hard power cut to the database, so take a backup once you are back in.

#### Already protected

- 4 GB swap file is active and survives reboots
- Both containers restart automatically after a reboot — production comes back on its own
- `.dockerignore` cut the build upload from about 1.2 GB down to 41 MB

## Worth doing eventually

Stop building on the production server altogether. The repository is public, so GitHub Actions can build the ARM64 image for free and push it — the server would then only need:

```
docker compose -f docker-compose.vm.yml pull  
docker compose -f docker-compose.vm.yml up -d
```

That removes this entire class of problem permanently.